Day 03 NLP Experiments Report

N-gram Language Models, LSTM Gradient Analysis, and Transformer Representations

Day 03 Report

Topics Covered:

- N-gram Language Models
- BERT/Transformer Embeddings
- LSTM Gradient Flow Analysis
- Text Similarity Comparison
- t-SNE Visualization of Embeddings

1. Introduction

Natural Language Processing (NLP) has evolved from simple statistical approaches to advanced deep learning architectures. Traditional NLP techniques such as N-grams and TF-IDF rely mainly on word frequency and local context, while modern transformer-based models understand deeper semantic relationships by considering the complete context of a sentence.

In this day's experiments, different NLP approaches were implemented and compared. The tasks started with traditional language modeling using N-grams, moved towards neural sequence modeling using LSTM, and finally explored transformer-based contextual representations.

The experiments demonstrate the limitations of traditional methods and the advantages of modern contextual embedding techniques.

2. Objectives

The main objectives of Day 03 were:

- Understand probability-based language modeling using N-grams.
- Generate contextual word embeddings using transformer models.

- Analyze gradient behavior in recurrent neural networks.
- Compare different text representation techniques using cosine similarity.
- Visualize static and contextual embeddings using t-SNE.

Task 1: N-gram Language Model (Bigram and Trigram)

Objective

The objective of this task was to build a simple statistical language model using Bigram and Trigram approaches.

N-gram models predict the probability of a word based on previous words.

A:

Bigram model

Uses one previous word:

Example:

I love

love NLP

Probability:

P(word2 | word1)

Trigram model

Uses two previous words:

Example:

I love NLP

Probability:

P(word3 | word1, word2)

Methodology

The following steps were performed:

1. Loaded the text corpus.
2. Converted text into lowercase.
3. Tokenized the corpus.
4. Created vocabulary.
5. Counted bigram and trigram occurrences.
6. Calculated probability distributions.
7. Generated predictions.

Implementation

The model was implemented using:

- Python dictionaries
- Token frequency counting
- Probability calculation

Observation

The Bigram model only considers one previous word, therefore it has limited context understanding.

The Trigram model performs better because it considers two previous words and captures more information.

However, both models struggle with long-range dependencies because they only consider a fixed amount of previous words.

Task 2: Transformer Embeddings Using BERT

Objective

The objective of this task was to understand contextual word embeddings using a pretrained transformer model.

Unlike traditional embeddings, transformers generate different representations for the same word depending on its surrounding context.

Experiment Setup

The ambiguous word:

bank

was tested in two different contexts.

Sentence 1:

I deposited money in the bank.

Meaning:

Financial institution

Sentence 2:

I sat near the bank of the river.

Meaning:

River side

Methodology

Steps:

1. Tokenized sentences using transformer tokenizer.
2. Added special tokens:

[CLS]

[SEP]

3. Generated embeddings using pretrained transformer model.
4. Compared vector representations.

Observation

The same word "bank" receives different embeddings because the transformer considers surrounding words.

For example:

money + bank

creates a financial meaning.

While:

river + bank

creates a geographical meaning.

This demonstrates the advantage of contextual embeddings.

Task 3: LSTM Gradient Flow Analysis

Objective

The objective was to analyze how gradients flow through an LSTM network during backpropagation.

Recurrent networks can suffer from the vanishing gradient problem, where earlier tokens receive extremely small gradient values.

Model Architecture

Implemented model:

SimpleLSTM

Input size:

10

Hidden size:

20

Output:

1

Architecture:

Input Sequence

↓

LSTM

↓

Linear Layer

↓

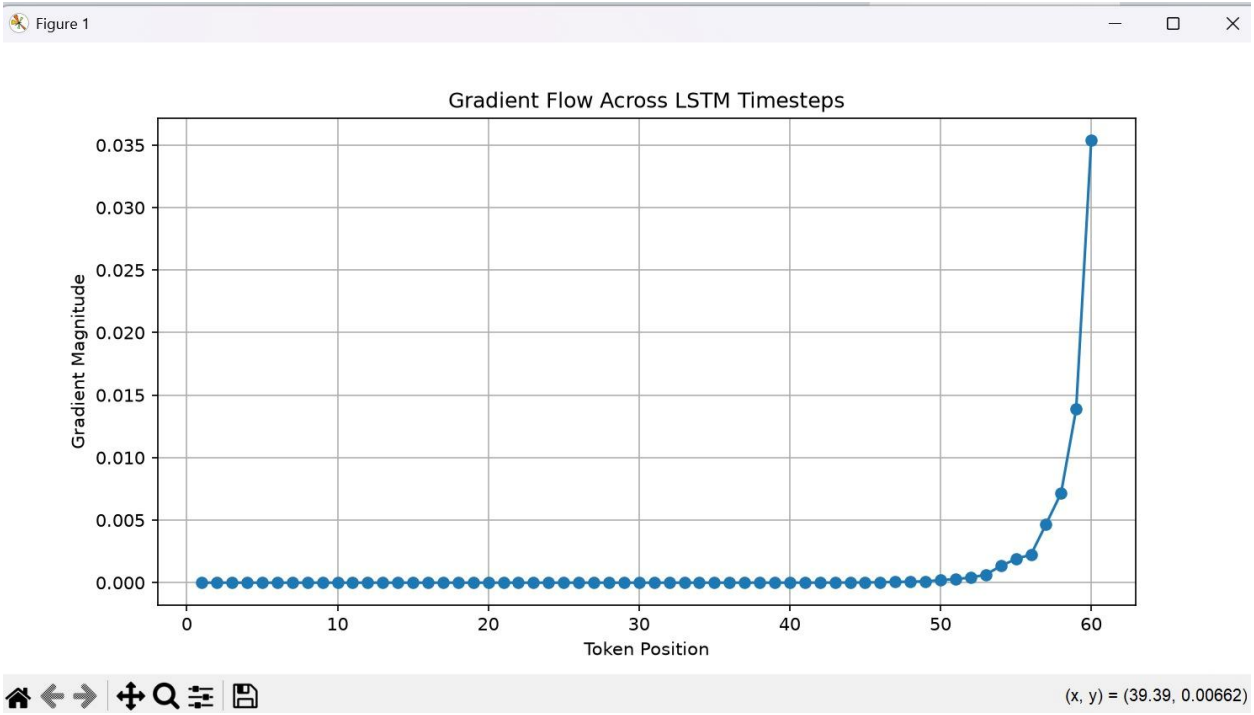
Prediction

Methodology

Steps performed:

1. Created random input sequence.
2. Passed sequence through LSTM.
3. Calculated prediction.
4. Computed loss.
5. Applied backward propagation.
6. Measured gradient magnitude for each token.

Output



Observation

The gradient values increased for later tokens and were extremely small for earlier tokens.

This shows that information from earlier parts of the sequence may not contribute effectively during learning.

This problem is known as:

Vanishing Gradient Problem

LSTM improves this issue compared to traditional RNNs but does not completely eliminate it.

Task 4: Similarity Comparison

Objective

The objective was to compare different text representation techniques using cosine similarity.

Methods compared:

1. TF-IDF
2. Static embedding (Word2Vec style)
3. N-gram representation

4. Transformer embeddings

Test Case 1: Synonym Similarity

Sentences:

Sentence 1:

The movie was very good.

Sentence 2:

The film was very excellent.

Expected:

Words:

movie ≈ film

good ≈ excellent

should have high similarity.

Test Case 2: Polysemy Similarity

Sentences:

Sentence 1:

I deposited money in the bank.

Sentence 2:

I sat near the bank of the river.

The word:

bank

has two different meanings.

Results

Final Comparison Table

Test	TF-IDF	Word2Vec (Static)	N-Gram	Transformer
Synonym Test	0.4316	0.6000	0.4444	0.9188
Polysemy Test	0.2967	0.3651	0.3443	0.4729

Observation

Transformer embeddings achieved higher similarity in synonym detection because they understand semantic relationships.

Traditional methods mainly depend on word overlap and frequency.

For polysemy cases, transformer models perform better because they use contextual information.

Task 5: t-SNE Visualization

Objective

The objective was to visualize the difference between static and contextual embeddings.

The ambiguous word:

bank

was selected.

Dataset

Financial context:

I deposited money in the bank.

The bank approved my loan.

She works at a bank.

River context:

I sat near the bank of the river.

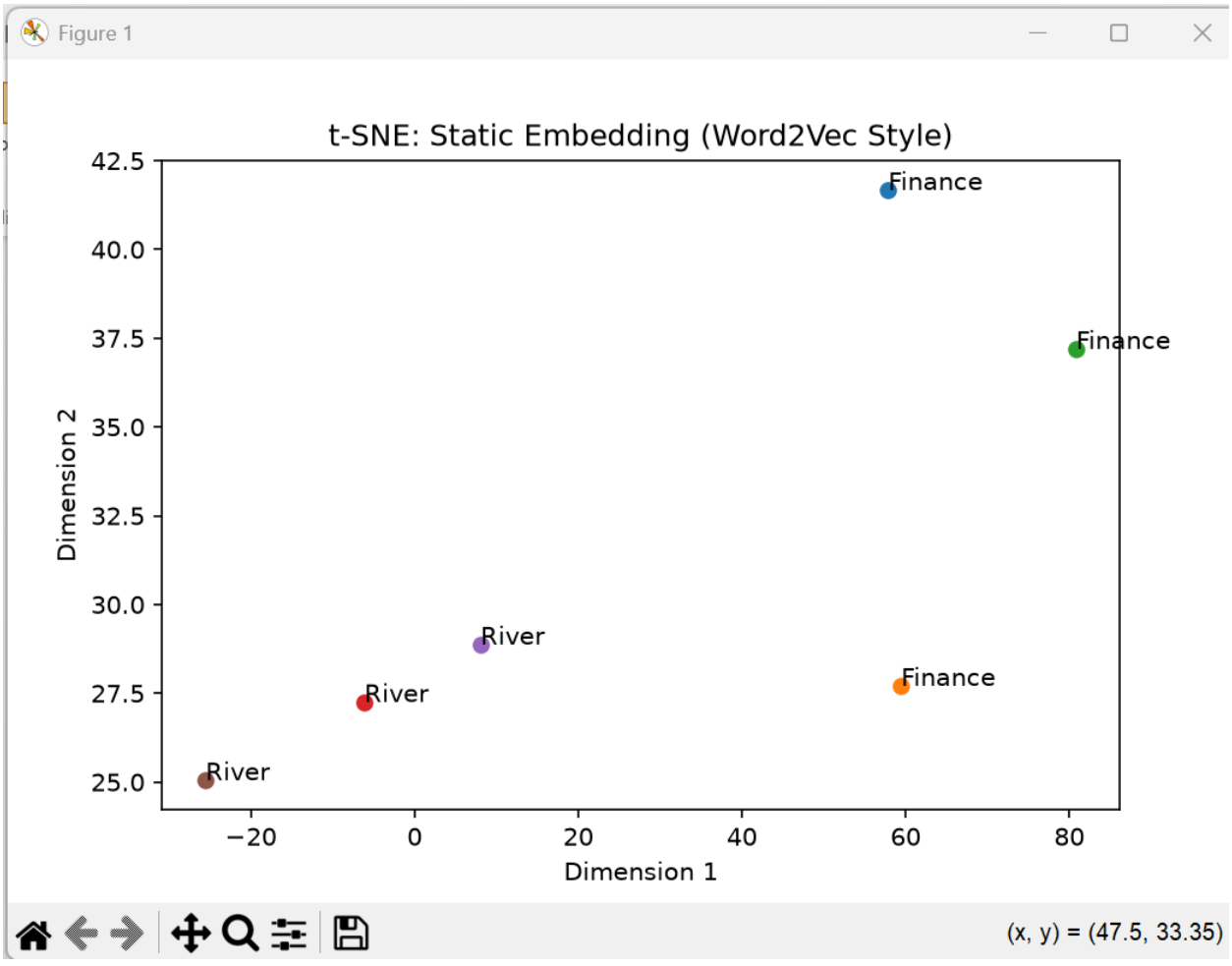
The boat reached the river bank.

We walked along the bank of the stream.

Methodology

Steps:

1. Generated static embeddings.
2. Generated transformer embeddings.
3. Applied t-SNE dimensionality reduction.
4. Converted embeddings into 2D visualization.



Observation

Static embeddings assign one representation to a word.

Therefore:

bank(finance)

bank(river)

remain similar.

Transformer embeddings generate different vectors according to context.

Therefore:

bank(finance)

and

bank(river)

form different clusters.

Overall Comparison

Method	Strength	Limitation
N-gram	Simple and interpretable	Limited context
TF-IDF	Good for keyword importance	No semantic understanding
Word2Vec	Captures word relationships	Same vector for different meanings
LSTM	Handles sequential information	Gradient issues
Transformer	Context-aware and powerful	More computationally expensive

---

Conclusion

The experiments demonstrate the progression of NLP techniques from simple statistical approaches to advanced transformer-based models.

Traditional methods such as N-grams and TF-IDF are useful for basic language analysis but fail to capture deeper meanings.

LSTM networks improve sequence modeling but face gradient-related challenges.

Transformer models provide powerful contextual understanding and are capable of handling ambiguity, semantic similarity, and complex language relationships.

Overall, transformer-based approaches provide the most effective representation of natural language.